# APP_CONTA — MANUAL INTEGRAL DEL SISTEMA

Guía completa de arquitectura, stack tecnológico, modelo de datos, API y frontend PWA

| | |
|---|---|
| **Proyecto:** App_Conta (Control Financiero del Hogar) | **Entorno:** Python 3.11 + Gestor 'uv' |
| **Arquitectura:** SPA/PWA Local + API REST FastAPI | **Base de Datos:** SQLite Relacional ('conta.db') |
| **Estado de Calidad:** 136 Tests Verdes (100% Passing) | **Cobertura Global:** 92% (Core Services al 94%) |
| **Migraciones:** Alembic (4 revisiones versionadas) | **Diseño UI:** Fintech Obsidian/Slate (Vanilla JS/CSS) |

## 1. ¿Qué es App_Conta y qué problema resuelve?

App_Conta nace para sustituir de forma definitiva el libro de control familiar en Excel (**OCTUBRE.xlsx** de 14 hojas) por una solución moderna, visual y rápida, ejecutable en red local (LAN) o en un servidor personal (VPS), permitiendo que ambos cónyuges utilicen la misma aplicación desde sus teléfonos móviles y desde el ordenador con sincronización instantánea.

A diferencia de las apps bancarias comerciales (que solo muestran saldos brutos) o las apps de finanzas genéricas (que fuerzan suscripciones y suben tus datos bancarios a servidores desconocidos), App_Conta se rige bajo el principio de **soberanía de datos y contabilidad por sistema de botes y sobres virtuales**.

### Los 5 Principios Fundacionales de la Aplicación

• **Principio I — Las deudas se modelan como entidades, no como signos:** Un préstamo, un anticipo o una cantidad pendiente no es un saldo negativo ficticio en una cuenta bancaria. Es una entidad con deudor, concepto, fecha y control estricto de cobros parciales o totales.

• **Principio II — Los saldos son siempre derivados por construcción:** El saldo de una cuenta o de un sobre NUNCA se escribe ni se edita a mano. Se calcula automáticamente sumando el saldo inicial más todos los movimientos vivos no eliminados. Esto hace matemáticamente imposible el descuadre contable.

• **Principio III — Aritmética exacta en céntimos enteros (Dinero seguro):** Los ordenadores sufren el problema de punto flotante de IEEE 754 (donde 0.1 + 0.2 = 0.30000000000000004). En App_Conta, TODO el dinero se almacena y calcula internamente como **enteros de céntimos** (ej. 30,50 € = 3050). Se usa Decimal con redondeo ROUND_HALF_UP únicamente en las fronteras.

• **Principio IV — Base de datos local, portable y sin dependencias:** Toda la contabilidad reside en un único archivo SQLite (conta.db). Hacer una copia de seguridad es tan fácil como copiar ese archivo. Si formateas el PC o cambias de máquina, te llevas tu contabilidad intacta.

• **Principio V — Cero Nube obligatoria y Privacidad Absoluta:** La app funciona 100% aislada de internet. No requiere APIs bancarias externas, no comparte tus gastos con terceros y no tiene trackers ni telemetría.

## 2. Stack Tecnológico: ¿Con qué está construida y por qué?

Se seleccionó una combinación tecnológica de alto rendimiento, bajo consumo de memoria y nula necesidad de configuración pesada:

| Tecnología | Versión | Rol en el Sistema | ¿Por qué se eligió? (Ventaja clave) |
|---|---|---|---|
| **Python** | `3.11.x` | Backend & Reglas contables | Tipado estricto, gran ecosistema matemático y rendimiento un 25% superior a versiones previas. |
| **uv** (Astral) | `0.5+` | Gestor de paquetes y venv | Escrito en Rust. 10x más rápido que pip. Bloqueo determinista en uv.lock. |
| **FastAPI** | `0.115+` | Servidor API REST HTTP | Asíncrono, auto-documentación OpenAPI (/docs) y validación de tipos vía Pydantic. |

| | | | |
|---|---|---|---|
| **Uvicorn** | `0.32+` | Servidor web ASGI | Servidor de producción ultrarrápido con soporte de sockets y binding en 0.0.0.0. |
| **SQLAlchemy** | `2.0+` | ORM Relacional de BD | Modelado tipado con Mapped. Claves foráneas reales e integridad referencial. |
| **SQLite** | `3.x` | Motor de Base de Datos | Embebido en fichero conta.db. Cero mantenimiento, transacciones ACID reales y máxima velocidad. |
| **Alembic** | `1.14+` | Migraciones de esquema | Evoluciona las tablas sin perder datos. Cada cambio tiene script de subida (upgrade) y bajada. |
| **OpenPyXL** | `3.1+` | Lector del Excel real | Lectura de las 14 hojas de OCTUBRE.xlsx en modo data_only para importar fórmulas. |
| **Vanilla PWA** | `ES Modules` | Frontend / Interfaz de usuario | Cero dependencias de Node.js, Webpack o React. Carga instantánea, limpia y ultra-ligera. |
| **Service Worker** | `v7 (sw.js)` | Caché y soporte móvil | Permite instalar la web como App nativa en iOS/Android y agiliza la navegación local. |
| **Pytest + Cov** | `8.x / 6.x` | Suite de pruebas automáticas | 136 tests ejecutables en <20s. Mide cobertura real de código línea a línea. |
| **Ruff** | `0.8+` | Linter y formateador | Reemplaza a flake8, black e isort con velocidad instantánea en Rust. |

# 3. Estructura de Archivos: ¿Dónde está cada cosa?

El código sigue una arquitectura limpia en 3 capas desacopladas: **Modelos (datos)** → **Servicios (lógica financiera pura)** → **API (controladores HTTP)** → **Frontend (PWA)**.

| Directorio / Fichero | Función y Contenido |
|---|---|
| `app_conta/db.py` | Motor SQLite. Configura engine, SessionLocal y la dependencia get_db para FastAPI. |
| `app_conta/main.py` | Punto de entrada de FastAPI. Registra routers, sirve la PWA en /static, auto-crea el Hogar único y muestra la IP local en consola. |
| `app_conta/models/` | **Capa de Datos:** Los 8 modelos SQLAlchemy (hogar.py, cuenta.py, movimiento.py, bote.py, categoria.py, auditoria.py, deuda.py, gasto_recurrente.py, importacion.py). |
| `app_conta/services/` | **Capa de Inteligencia Contable:** Funciones independientes de la web. Cálculos matemáticos, validaciones, auditoría y conciliación. |
| `app_conta/api/` | **Capa de Exposición REST:** Endpoints HTTP (cuentas.py, inversiones.py, movimientos.py, botes.py, conciliacion.py, deudas.py, prevision_gastos.py, importacion.py). |
| `app_conta/static/` | **Frontend PWA:** index.html (shell único), styles.css (diseño fintech), sw.js (Service Worker) y manifest.webmanifest. |
| `app_conta/static/js/` | **Lógica de Cliente:** api.js (cliente fetch), vistas.js (registro de pantallas e iconos SVG), nav.js (menú) y app.js (router y eventos). |
| `app_conta/static/js/views/` | **Pantallas Individuales:** inicio.js, cuentas.js, inversiones.js, tarjetas.js, tarjetas-regalo.js, gastos-tarjetas.js, ahorro.js, botes.js, prevision-gastos.js, deben.js, categorias.js, importar.js. |
| `migrations/versions/` | **Control de Versiones de BD:** Los 5 scripts Alembic que construyen la base de datos desde cero sin romper datos. |
| `tests/` | **Pruebas de Calidad:** Tests unitarios (tests/unit/), de contrato API (tests/api/) y de integración end-to-end (tests/integration/). |
| `levantar.bat` / `iniciar_servidor.bat` | Scripts directos de Windows para arrancar uvicorn en red local mediante uv run en un solo clic. |

# 4. Modelo de Datos y Base de Datos Relacional

La base de datos SQLite almacena la información estructurada mediante claves foráneas e índices para máxima velocidad. A continuación se detalla cada entidad y para qué sirve:

• **Hogar (hogares):** El contenedor maestro del sistema. Todos los datos (cuentas, botes, deudas) cuelgan de un hogar_id. Permite en el futuro multi-hogar sin cambiar el esquema.

• **Cuenta (cuentas):** Representa los depósitos de dinero real. Tipos: banco (cuentas corrientes), tarjeta (tarjetas de crédito/débito y regalo), efectivo (caja física) o inversion. Almacena nombre, entidad bancaria (Caixa, Deutsche, BBVA, Revolut...) y saldo_inicial_centimos.

• **Movimiento (movimientos):** Cada transacción individual. Registra fecha, importe_centimos (con signo: positivo = ingreso, negativo = gasto), descripción, cuenta_id (obligatorio), bote_id (opcional, sobre al que se imputa), categoria_id (opcional) y flag eliminado (borrado lógico para preservar auditoría).

• **Bote (botes):** Los sobres virtuales donde se guarda o compromete el dinero. Tipos: ahorro (fondos para hipotecas o reservas), recurrente (gastos previsibles), proyecto o ajeno. Su saldo es siempre la suma de los movimientos asignados a él.

• **Categoria (categorias):** Etiquetas analíticas opcionales para clasificar consumos (ej. Supermercado, Gasolina, Restaurantes). Cuenta con selector de color hexadecimal y cálculo de totales por periodos de fechas.

• **AuditoriaMovimiento (auditoria_movimientos):** Caja negra de seguridad. Registra automáticamente cada creación, edición, eliminación o reasignación de bote con los valores anteriores y nuevos en formato JSON para trazabilidad total.

• **Deuda (deudas):** Módulo 'Dinero que me deben'. Almacena nombre del deudor, concepto, importe inicial, importe cobrado acumulado, fecha, fecha límite y estado (pendiente, parcial o saldado). Permite registrar cobros con generación opcional de ingreso en cuenta.

• **Inversion (inversiones):** Módulo de 'Inversiones y Patrimonio'. Carteras de fondos indexados, acciones/ETFs, renta fija/letras, pensiones y cripto. Registra capital aportado, valor actual de mercado, plusvalía (+€) y rentabilidad ponderada (ROI %).

• **GastoRecurrente (gastos_recurrentes):** Planificación anual de tributos y recibos (IBI, basuras, seguros, sellos). Guarda importe anual presupuestado, cuota mensual fija (total 176,54 €/mes), pagado real acumulado y notas de cobro.

• **PrevisionGastosConfig (prevision_gastos_config):** Configuración del bote de gastos anuales. Controla el año activo y los meses transcurridos/aportados (1 a 12 meses) para calcular la provisión teórica acumulada.

• **Importacion (importaciones):** Historial de importaciones del Excel OCTUBRE.xlsx. Guarda nombre, huella digital SHA-256 para evitar duplicados y resumen con avisos de filas dudosas detectadas.

# 5. Capa de Servicios: La Lógica Contable y Fórmulas

En una arquitectura profesional, los controladores de la API no deben contener cálculos matemáticos. Toda la inteligencia reside en app_conta/services/, donde se implementan las fórmulas contables maestras:

**Fórmula Maestra del 'Disponible Real' y Conciliación (services/conciliacion.py)**

**Diferencia = Saldo Total Cuentas** – **Saldo Inicial Total** – **(Saldo Total Botes + Sin Asignar)**

• **Estado 'Cuadrado' (Verde):** La diferencia es exactamente 0.00 €. Todo el dinero físico en bancos y caja coincide con la suma de los sobres comprometidos más el dinero libre.
• **Estado 'Revisar' (Rojo):** Existe un desfase. La pantalla avisa del importe exacto sin cuadrar.
• *Nota de diseño:* Este servicio SOLO LEE la base de datos (Principio II); jamás altera ningún registro.

## Segregación Financiera: Dinero en Caja vs. Dinero en Bancos

A petición expresa de las reglas del hogar, se implementó una estricta segregación matemática:
• **Dinero en Caja:** Suma *exclusivamente* el dinero físico en metálico (cuentas con tipo == 'efectivo'). No incluye bancos.
• **Dinero en Bancos:** Suma las cuentas corrientes y depósitos bancarios (cuentas con tipo == 'banco').
• **Dinero Disponible (Hero principal):** Es el total líquido real del hogar: Dinero en Caja + Dinero en Bancos.

## Módulo de Previsión de Gastos y Bote Anual (services/prevision_gastos.py)

Equivalente a la hoja **GASTOS 2026** de tu Excel. Los gastos anuales (IBI Cañete, IBI Piso, IBI Córdoba, Basura, Seguros, Sellos) se dividen en 12 mensualidades exactas. El sistema calcula:
1. **Cuota Mensual Fija a Guardar:** Suma de las mensualidades (actualmente **176,54 €/mes**).
2. **Provisión Acumulada:** (Importe Anual * Meses Aportados) / 12.
3. **Saldo del Bote:** Provisión Acumulada – Facturas Reales Pagadas.
Si el saldo es positivo, el bote está en **Superávit** (fondos suficientes guardados). Si es negativo, marca **Déficit**.

# 6. Catálogo de la API REST (Rutas y Endpoints)

FastAPI expone los endpoints bajo el prefijo común /api/v1, comunicando el backend con la interfaz web mediante JSON:

| Endpoint HTTP | Método | Descripción y Funcionalidad |
|---|---|---|
| `/api/v1/conciliacion` | **GET** | Devuelve el estado global, dinero en cuentas, dinero en botes, sin asignar y cuadre. |
| `/api/v1/cuentas` | **GET / POST** | Listado con saldo derivado de cuentas y alta de nueva cuenta (banco, efectivo, tarjeta). |
| `/api/v1/cuentas/{id}` | **PATCH / DEL** | Edición de datos (el saldo se ignora por Principio II) y borrado protegido con archivado. |
| `/api/v1/movimientos` | **GET / POST** | Consulta con filtros (cuenta, bote, categoría, fechas) y registro de nuevo ingreso/gasto. |
| `/api/v1/movimientos/{id}` | **PATCH / DEL** | Edición de movimiento con registro en auditoría y borrado lógico (desaparece de listados). |
| `/api/v1/movimientos/{id}/bote` | **PATCH** | Reasignación directa de sobre/bote con auditoría de cambio atómica. |
| `/api/v1/botes` | **GET / POST** | Consulta de botes con saldos derivados, porcentajes de progreso y alta de nuevos sobres. |
| `/api/v1/botes/{id}` | **GET / DEL** | Extracto individual con movimientos del sobre y borrado (libera fondos a sin asignar). |
| `/api/v1/deudas` | **GET / POST** | Listado de dinero que me deben con KPIs y registro de nuevos préstamos. |
| `/api/v1/deudas/{id}/cobro` | **POST** | Registra un cobro parcial o total, con opción de crear ingreso contable automático en cuenta. |
| `/api/v1/prevision-gastos` | **GET / POST** | Resumen del bote de gastos anuales, cuota fija y listado comparativo de recibos. |
| `/api/v1/prevision-gastos/config` | **PATCH** | Actualiza el stepper del mes de provisión aportado (1 a 12 meses). |

| | | |
|---|---|---|
| `/api/v1/prevision-gastos/{id}/pagar` | **POST** | Registra el importe de una factura real cobrada (IBI, Seguro...) actualizando el saldo. |
| `/api/v1/importacion/previsualizar` | **POST** | Analiza OCTUBRE.xlsx en memoria, calcula recuentos y detecta huella SHA-256 sin tocar BD. |
| `/api/v1/importacion/aplicar` | **POST** | Aplica de forma transaccional el volcado contable a la base de datos. |

# 7. Frontend PWA: Interfaz, Navegación y Vistas

El frontend se construyó bajo la filosofía **Vanilla ES Modules**: código limpio, modular y sin compiladores pesados. Cada pantalla es un módulo JavaScript independiente en static/js/views/ que se auto-registra mediante vista(nombre, renderFn).

## Arquitectura de Navegación Consolidada (5 Pestañas)

Para erradicar la saturación horizontal y la barra de desplazamiento ('línea para desplazar'), el menú superior se consolidó en **5 secciones maestras** de ancho fijo holgado (~580px en contenedor de 1080px):

- **1. ■ Resumen (#/inicio):** Panel global del hogar. Muestra el Dinero Disponible líquido (Caja + Bancos), el badge vivo de conciliación, las tarjetas estadísticas y los botes destacados.
- **2. ■ Cuentas y Tarjetas (#/cuentas):** Hub financiero agrupado. Dispone de subnavegación por píldoras para alternar entre:
  - [ ■ Cuentas y Depósitos ]: Agrupadas por bancos (Caixa, Deutsche Bank, BBVA, Revolut, Otros).
  - [ ■ Inversiones ]: Carteras de fondos, ETFs, renta fija, pensiones y cripto con plusvalía y ROI %.
  - [ ■ Tarjetas Bancarias ]: Tarjetas de crédito/débito digitales con chip visual y saldo dispuesto.
  - [ ■ Tarjetas Regalo ]: Tarjetas prepago para compras (Amazon, Carrefour, Decathlon...).
  - [ ■■ Gastos Tarjetas ]: Movimientos y compras imputadas a cada tarjeta.
- **3. ■ Ahorro y Botes (#/ahorro):** Hub de previsión futura y sobres de gasto. Subnavegación por píldoras entre:
  - [ ■ Fondos de Ahorro e Hipotecas ]: Amortización de hipotecas, reservas futuras y extracto individual.
  - [ ■ Sobres y Botes ]: Sobres para vacaciones, regalos o proyectos con barra de progreso.
  - [ ■ Previsión Gastos Anuales ]: Control de IBI, basuras, seguros, sellos y cuotas mensuales de 176,54 €.
  - [ ■ Dinero que me deben ]: Registro de préstamos a terceros y control de recuperaciones.
- **4. ■ Movimientos (#/movimientos):** El libro diario contable. Registro cronológico con importes coloreados (+ verde / – rojo), selector de bote y categoría directamente en la fila y formulario rápido con control segmentado de Gasto/Ingreso.
- **5. ■ Más (Desplegable flotante):** Ventana glassmorphism para herramientas secundarias: **Categorías** (clasificación con colores) e **Importar Excel** (lector de OCTUBRE.xlsx).

## Sistema de Diseño Fintech (Fintech Design System)

- **Tipografía:** Plus Jakarta Sans para textos y navegación; JetBrains Mono para importes tabulares.
- **Iconografía:** Sustituidos los emojis por un set completo de **iconos vectoriales SVG limpios** en vistas.js.
- **Paleta Obsidian / Slate:** Fondos oscuros translúcidos con efecto *glassmorphism* (desenfoque de 20px), sombras sutiles y tarjetas elevadas.

# 8. Ciclo de Vida: ¿Qué pasa cuando abres la aplicación?

Explicación técnica paso a paso desde que pulsas levantar.bat hasta que interactúas en el móvil:

**1. Ejecución del script:** El archivo levantar.bat ejecuta uv run uvicorn app_conta.main:app --host 0.0.0.0 --port 8000. El gestor uv localiza el entorno virtual .venv garantizando que todas las librerías están presentes.

**2. Arranque de FastAPI:** Se ejecuta el evento lifespan de main.py. Se conecta a conta.db. Si la base de datos está vacía, crea automáticamente el **Hogar único** con ID '00000000-0000-0000-0000-000000000001'.

**3. Publicación en LAN:** FastAPI detecta las tarjetas de red del PC e imprime en consola la URL local (http://127.0.0.1:8000) y la URL para móviles (ej. http://192.168.1.50:8000). Abre automáticamente el navegador.

**4. Carga del Frontend:** El navegador solicita /, que devuelve static/index.html. El HTML descarga los estilos styles.css, las Google Fonts y el script de arranque app.js como módulo ES.

**5. Peticiones concurrentes:** La vista inicio.js realiza llamadas en paralelo mediante Promise.all a /api/v1/conciliacion, /api/v1/cuentas, /api/v1/botes, /api/v1/deudas y /api/v1/prevision-gastos.

**6. Inyección de Sesión BD:** Cada endpoint FastAPI recibe la sesión SQLite mediante la dependencia Depends(get_db), que abre una transacción segura y garantiza el close() al finalizar la respuesta JSON.

**7. Renderizado reactivo:** El JavaScript inyecta en <main id='vista'> el HTML final con los importes formateados (signos, separador de miles y decimales en coma). Al pulsar un botón, la delegación global de eventos captura la acción y refresca la pantalla sin recargar la página.

# 9. Calidad y Tests Automatizados: La Red de Seguridad

Siguiendo el **Principio III de la Constitución Contable (TDD Obligatorio)**, ninguna regla financiera se programó sin haber escrito primero una prueba automatizada que fallara. Hoy la suite cuenta con **136 tests en verde**:

| Módulo de Pruebas | Fichero de Tests | Tests | Aspecto que Verifica |
|---|---|---|---|
| Aritmética y Dinero | `tests/unit/test_dinero.py` | **21** | Conversión exacta euros-céntimos, redondeo ROUND_HALF_UP, rechazo de importe 0.00 y signos. |
| Cuentas y Movimientos | `tests/api/test_cuentas_movimientos.py` | **19** | CRUD de cuentas, cálculo de saldo derivado, filtros de movimientos y borrado lógico. |
| Flujo US2 Integración | `tests/integration/test_us2_flujo.py` | **3** | Cuenta 1000 € + 3 movs → saldo exacto 2449.50 € → recálculo y auditoría en DB. |
| Conciliación y Cuadre | `tests/unit/test_conciliacion.py` | **9** | Cuadre en cero, detección de desfases, saldos negativos e ignorado de archivados. |
| Botes y Sobres | `tests/unit/test_botes.py` | **22** | Suma de movimientos asignados, objetivos conseguidos, reasignación atómica entre sobres. |
| Categorías con Fechas | `tests/unit/test_categorias.py` | **18** | Totales por periodo desde/hasta, nombres únicos, protección de borrado si tiene movimientos. |
| Importador OCTUBRE.xlsx | `tests/integration/test_importador.py` | **26** | Mapeo sintético + fichero real (20 cuentas, 28 botes, 215 movs), huella SHA256 y conciliación. |
| Dinero que me deben | `tests/unit/test_deudas.py` | **6** | Registro de préstamos, cobros parciales/totales y estados (pendiente, parcial, saldado). |
| Previsión de Gastos | `tests/unit/test_prevision_gastos.py` | **10** | Semilla de 16 gastos, cuota mensual de 176.54 €, provisión acumulada y registro de pagos. |
| Arranque y Migraciones | `tests/integration/test_arranque.py` | **2** | Ejecución de migraciones Alembic desde BD vacía y verificación del Hogar único. |

# 10. Guía Práctica de Uso y Mantenimiento

Instrucciones directas para operar la aplicación en el día a día sin complicaciones técnicas:

- **Arrancar la aplicación:** Hacer doble clic en levantar.bat. Se abrirá la consola de comando y el navegador cargará la app en http://localhost:8000.
- **Abrir la app en los teléfonos móviles:** Conectar el móvil a la misma red WiFi del hogar y abrir en Safari/Chrome la dirección IP que aparece en la ventana de consola (ejemplo: http://192.168.1.50:8000).
- **Instalar como App en el móvil:** En Chrome (Android) pulsar en 'Añadir a la pantalla de inicio'. En Safari (iPhone) pulsar 'Compartir' → 'Añadir a pantalla de inicio'. La app se abrirá sin barras de navegador.
- **Hacer una Copia de Seguridad manual:** Basta con cerrar la app y copiar el archivo conta.db a una memoria USB, carpeta de Google Drive o disco externo. Contiene el 100% de tus datos contables.
- **Ejecutar las pruebas automáticas:** Abrir PowerShell en la carpeta del proyecto y ejecutar uv run pytest. Comprueba que los 136 tests pasan en verde.
- **Comprobar estilo y formato del código:** Ejecutar uv run ruff check para verificar que no hay errores de código o sintaxis.

**Certificación de Integridad del MVP (Hito Octubre 2026)**

Este documento refleja el estado exacto del código registrado en el commit **129d4e1** de la rama master. La aplicación reproduce la totalidad de las reglas contables del Excel original con total precisión matemática en céntimos, ofreciendo una experiencia moderna, segura, autónoma y multi-dispositivo.